

IM05: Report on Kaggle Challenge

Felipe Scherer Vicentin

May 2025

Contents

1	Introduction	2
2	Dataset Description	2
2.1	Reading data	2
2.2	Data exploration	3
3	Classifier selection	4
4	Features selection	4
5	LV Segmentation	6
6	Selected models	7
7	Results	8
8	Conclusion	10

1 Introduction

Cardiovascular diseases remain a leading cause of death globally, underscoring the importance of accurate and early diagnosis. Advances in medical imaging, particularly cardiac magnetic resonance imaging (MRI), have enabled clinicians to non-invasively examine cardiac structure and function. However, interpreting these images remains a complex and time-consuming task that requires specialized expertise.

The presented challenge [2] invites students to develop machine learning models that can automatically classify cardiac pathologies using MRI scans of the heart at two critical phases: End Diastole (ED) and End Systole (ES). Each scan is accompanied by voxel-wise segmentations of the heart structures, namely the left and right ventricles and the myocardium, alongside basic metadata such as the patient's weight and height.

The goal of this challenge is to predict one of five possible diagnoses for each subject in the test set. This is a supervised classification problem where performance is evaluated mainly by accuracy.

This report documents the steps taken to build a competitive solution for this task. These include data exploration and preprocessing, handcrafted feature selection, segmentation of missing left ventricle segmentation, classifier selection, and performance evaluation. Inspired by previous approaches in the literature and adapted to the constraints of the provided data, the proposed solution balances personal intuition with algorithmic robustness.

2 Dataset Description

The dataset at hand consists of MRI scans of subjects. For each subject, 2 scans are provided: The heart at the end of systole (ES) and the heart at the end of diastole (ED). Also, for each scan, a segmentation is given. Each voxel can be one of these 4 categories:

- Background;
- Right Ventricle (RV);
- Myocardium (Myo);
- Left Ventricle (LV).

A table containing the height and weight of each individual is also provided. In total, we have 100 subjects for training and 50 for testing. There is only one difference between the training and the testing data: The LV segmentation on the test set is not given (it is set to the background class).

For the training data, the label of each subject's heart condition is provided. These labels can be one of the 5 following classes:

- Normal;
- Myocardial infarction;
- Dilated cardiomyopathy;
- Hypertrophic cardiomyopathy;
- Abnormal right ventricle.

Naturally, the labels are not provided in the test data, since this is the prediction task.

2.1 Reading data

All MRI and segmentation files provided are NIFTI format images (.nii files). To read it, we make use of the `nibabel` python package. Once read, this package returns an object with several attributes and methods, but we are interested in two of them: the `header` attribute and the `get_fdata` method. The former prints useful information and metadata about the file and the latter returns a 3-dimensional numpy array that contains the image itself.

With the goal of orderly representing the data given, I decided to use python's inheritance capabilities. In particular, I created a `MRI` class that inherits from `np.ndarray`. The only thing that is changed from the base class is the addition of an attribute `nii` that holds the initial nifti object returned by

nibabel. Representing the data this way, I can easily manipulate the MRI data (since it is a numpy array) and quickly access further metadata by using the `nii` attribute.

To organize the subjects, I decided to use a much simpler data structure: the `namedtuple`. It is analogous to the `struct` type in C. Within it, I am storing 6 attributes: `ed`, `ed_seg`, `es`, `es_seg`, `weight`, and `height`. Each of the 4 first attributes is an instance of the MRI class described previously. Weight and height are stored as floats. Finally, I normalized all MRI images to be in the range $[0, 1]$.

2.2 Data exploration

The first step I took when exploring the data was printing the information contained in the header of the Nifti images. Notably, the two most important attributes in the header are the dimension of the MRI image (`dim`) and the voxel dimension (`pixdim`). The former can be easily found by analyzing the shape of the numpy array of the MRI scan, so it is not useful. The latter will be extremely useful and will be discussed in Section 4.

After that, I decided to showcase a small video of the ED MRI of the first subject. To this end, a simple `video` function was coded. It uses `matplotlib` to simply successively show the heart slices. The first thing I noticed was that it might be a good idea to equalize the histograms of the MRI scans. However, since I still do not know which features I will use and how the model will be built, I left this thought for the future.

Next, I started to look for possible outliers. Firstly, I checked whether the MRI scans had the same dimensions. Unfortunately, the dimensions differed substantially. The differing width and height of scans may not be a significant problem because we can always resize images to a larger size without losing information (although we might lose resolution). The biggest problem at this point was the differing *depth* of scans. The number of heart slices varied between 6 and 18. How to correctly represent these differing depths? Maybe this is due to the heart size? Or maybe the resolution of the scan changed depending on the subject? Regardless of the cause, I decided, again, to leave this problem for the future. Fortunately, the shapes between ED and ES matched, meaning that the MRIs of the same subject had the same dimensions. Finally, I decided to plot histograms of the distribution of dimension size and a heatmap of the width and height of MRIs. It is possible to see that the average scan has dimensions $216 \times 256 \times 10$.

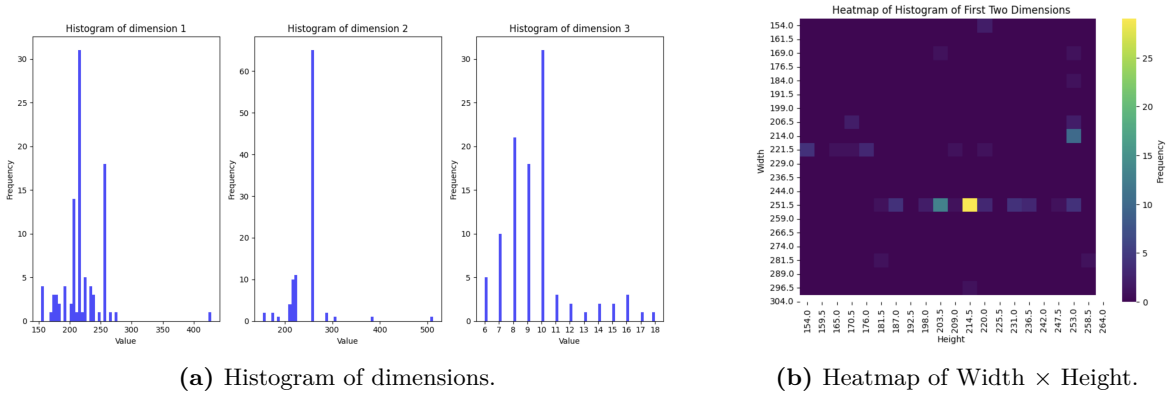


Figure 1: Analysis of dimensions distribution across MRI images.

Finally, I explored the metadata provided for the training set. Firstly, the weight and height of subjects appears to be stored as floating points respectively in kilograms and centimeters. Then, I checked the distribution of labels in our dataset to check for unbalanced classes. Luckily, all classes are balanced, so I will not have to worry about any biases regarding class distribution. After that, I searched for outliers in the metadata. Fig. 2 shows the histograms of height and weight. As we can see, no outliers appear to emerge from the data. No empty values were found in the training set. Lastly, I checked for NaNs in the testing data, but none were found. In theory, we *should not* have access to the test data, so I stopped my analysis of the test data here.

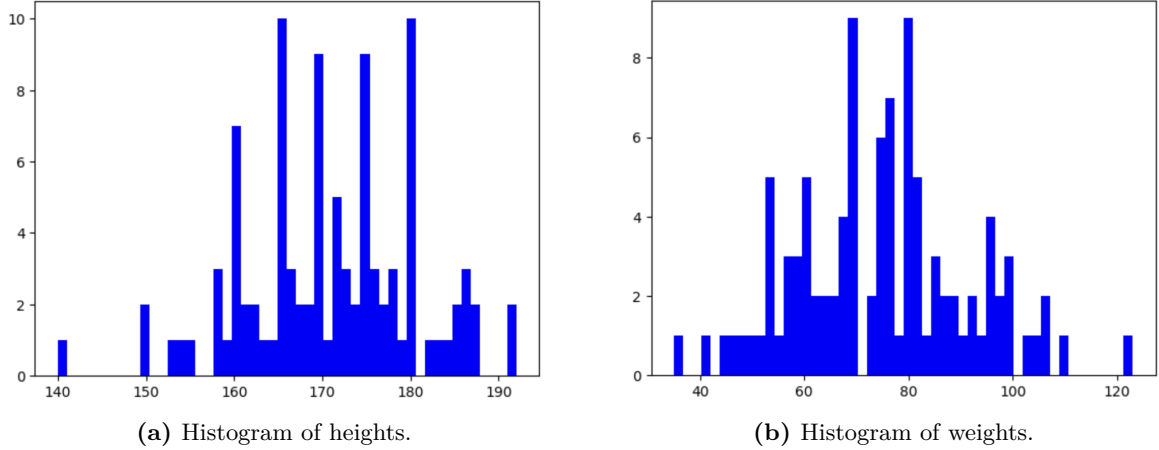


Figure 2: Analysis of metadata in the training set.

3 Classifier selection

Bernard *et al.* [1] conducted an analysis on different strategies to tackle segmentation and classification of a variation of this challenge. In particular, the testing data contained no segmentation whatsoever (instead of only the missing LV). So, the different participants had to come up with models to both segment and, then, classify the MRI scans.

Regarding the classification model comparison, the authors analyzed the 4 best accuracies in the test data. Tab. 1 shows the classification results. It is clear that the best performing architecture resolves around Random Forests (RFs), including the best model (with a 4% difference to the second best).

Authors	Architectures	Accuracy
Khened <i>et al.</i>	Random Forest	0.96
Cetin <i>et al.</i>	SVM	0.92
Isensee <i>et al.</i>	Random Forest	0.92
Wolterink <i>et al.</i>	Random Forest	0.86

Table 1: Results on the Classification Challenge

Khened *et al.* [4] used an RF classifier with 100 trees. But because our dataset is not huge, I decided to do a *grid search* with cross-validation in the space of possible classifiers. Using the `sklearn` library, I chose to vary the following parameters:

- `n_estimators`: Controls how many trees are used in the forest;
- `max_depth`: Sets the maximum depth of each tree (avoids overfitting).

4 Features selection

Given that the chosen classifier to train is a RF, we have to come up with meaningful features. Since Khened *et al.* obtained a great performance with their model, I decided to start with the same 10 features they defined in their model. These features are:

1. Weight;
2. Height;
3. Left Ventricle volume at End Diastole;
4. Left Ventricle volume at End Systole;
5. Right Ventricle volume at End Diastole;
6. Right Ventricle volume at End Systole;

7. Myocardium volume at End Systole;
8. Myocardium mass at End Diastole;
9. Ejection Fraction of Left Ventricle;
10. Ejection Fraction of Right Ventricle;

Features 1 and 2 are straightforward. For volume features, it would suffice to count the number of voxels that make up each segmentation. However, as previously discussed, the size of each MRI is different. So, simply counting the number of voxels is not enough. Thankfully, the Nifti images contain the voxel dimensions. Using this data, we are able to easily compute the volume of each structure by multiplying the number of voxels by the volume of each voxel. Thus, we obtain features 3-7.

Feature 8 is the mass of the myocardium. To find this quantity, it would suffice to multiply the volume of the myocardium by its density, which is approximately 1.05 g/cm^3 . However, as studied during the courses, Random Forests are built from many Decision Trees (DTs). Each DT computes the best splits using threshold algorithms. As such, *multiplying constants do not affect the result*. This means that for the DTs, the volume of the myocardium is equivalent to its mass because their difference is just a scaling factor. So, in our features, we used the Myocardium *volume* at End Diastole.

The two last features are the ejection fractions. They are computed from the volumes at ED and ES as follows:

$$\text{EF} = \frac{\text{Vol}(ED) - \text{Vol}(ES)}{\text{Vol}(ED)}.$$

This concludes all features used by Khened *et al.*

RF classifiers do not consider non-linear combinations of features, so Wolterink *et al.* [5] propose four other features:

11. The ratio between RV and LV volumes at ED;
12. The ratio between RV and LV volumes at ES;
13. The ratio between myocardial and LV volumes at ED;
14. The ratio between myocardial and LV volumes at ES.

After that, I decided to invent some features on my own. Using the subject's height, weight and ED segmentation, the next 4 features were computed:

15. Body-Mass Index;
16. Max Myocardium thickness;
17. Average Myocardium thickness;
18. Minimum Myocardium thickness.

The three last features were computed with the help of the *Euclidean distance transform*. This transform assigns to each voxel the shortest distance to the background. By applying it to the myocardium mask slice-wise, we extracted the minimum, average, and maximum distances within each slice, thereby quantifying the myocardial wall thickness in multiple ways.

Afterwards, I introduced additional non-linear features that capture ratios between key anatomical structures and time points. These include:

19. The ratio between myocardial and RV volumes at ED;
20. The ratio between myocardial and RV volumes at ES;
21. The ratio between myocardial maximum thickness and LV volume at ED;
22. The ratio between RV volumes at ES and ED;
23. The ratio between LV volumes at ES and ED;
24. The ratio between the left and right ejection fractions.

Finally, given that decision trees do not model linear combinations across features, I manually introduced a few such interactions. These help the model detect subtractive relationships between physiological metrics:

25. The LV volume minus the RV volume at ES;
26. The LV volume minus the RV volume at ED;
27. The RV volume at ED minus the RV volume at ES;
28. The LV volume at ED minus the LV volume at ES;
29. The myocardium volume at ED minus the myocardium volume at ES;
30. The left ejection fraction minus the right ejection fraction.

This brought the total feature count to 30.

5 LV Segmentation

Naturally, many of the features depend on the correct segmentation of the Left Ventricle. Back to Bernard *et al.* [1], for the segmentation task of the structures of the heart, the authors compiled the different strategies of the participants into a table, showing how well they performed. To this end, they used the Dice score and the Hausdorff distance. For simplicity, we will only consider the Dice score. Across all proposed models, the best Dice score for the LV segmentation was 0.968 for ED and 0.931 for ES. These metrics were obtained by Isensee *et al.* [3] using a combination of 2D and 3D U-Net architectures with a Dice loss.

Before going any further with implementing U-Nets, I played a video of some random scans of both ED and ES with their respective segmentation. Fig. 3 shows some random slices (frames) of these videos. One aspect of them immediately caught my attention: the left ventricle seems to always be inside the myocardium. Upon confirmation with Prof. Le Folgoc, this appears to be the case for *all* scans.

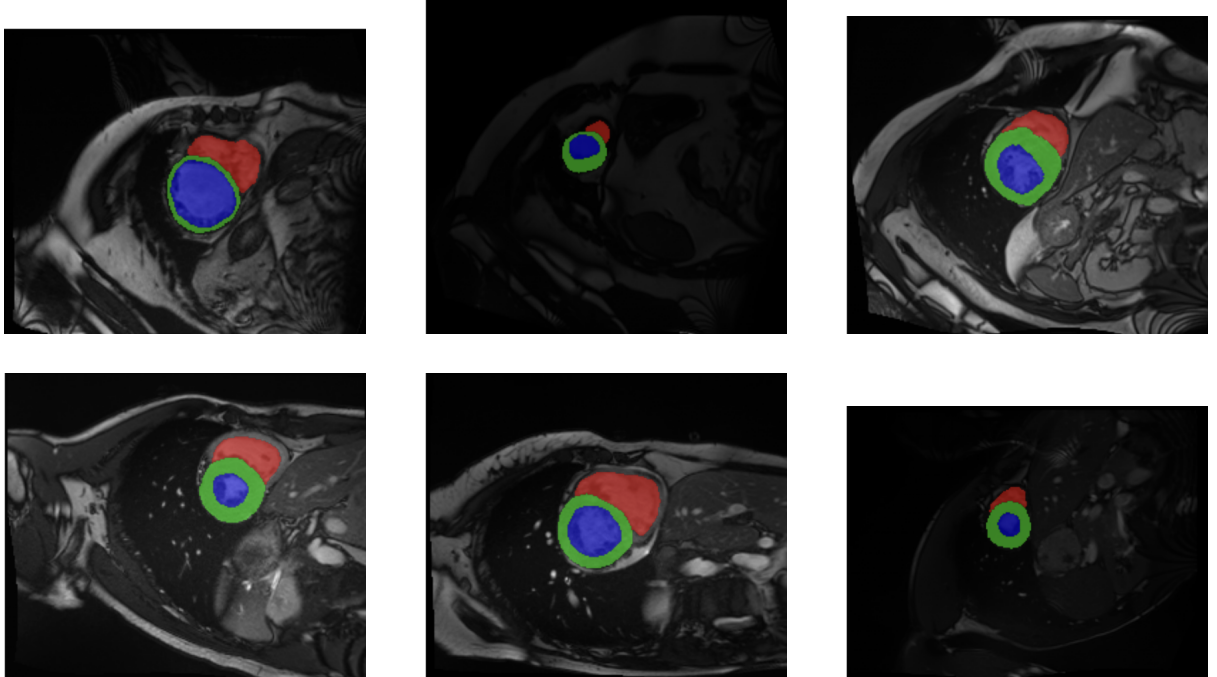


Figure 3: Random slices of scans of different subjects and moments (ED and ES). Red: RV; Green: Myo; Blue: LV.

Thus, the task of LV segmentation became substantially simpler. We just have to fill whatever is inside the myocardium as the left ventricle.

To this end, I used `scipy.ndimage's binary_fill_holes` function. It takes any binary image and fills holes in it. We can consider the myocardium mask as a binary image whose hole is precisely the LV. So, the first version of the LV segmentation algorithm would be as follows:

Algorithm 1: Naïve LV Segmentation

Input: Myocardium mask $\mathcal{M} = \{0, 1\}^{H \times W \times C}$
Output: LV mask $\mathcal{L} = \{0, 1\}^{H \times W \times C}$

- 1: **for** each slice $z = 1$ **to** C **do**
- 2: Take slice $\mathcal{M}_z \leftarrow \mathcal{M}[:, :, z]$
- 3: $\mathcal{L}_z \leftarrow \text{binary_fill_holes}(\mathcal{M}_z)$
- 4: $\mathcal{L}_z \leftarrow \mathcal{L}_z \oplus \mathcal{M}_z$ $\triangleright$ XOR operation to only keep the filled hole
- 5: **return** $\mathcal{L}_1 \cup \dots \cup \mathcal{L}_C$

As a metric to evaluate the segmentation of the ventricle, I used the *Dice score*. It is defined as follows:

$$\text{Dice}(X, Y) = \frac{|X \cap Y|}{|X| + |Y|},$$

where X would be the real segmentation (ground truth) and Y is what the algorithm was able to segment. Upon reviewing the dices, most subjects had 1.0 as its score, which means perfect segmentation. For the rest of subjects, they all had 0.0 as score, that is, completely wrong segmentation. When reviewing the slices that were not properly segmented, the problem became instantly clear: some slices had myocardium segmentations that were not closed, i.e., they did not form holes.

In order to circumvent this issue, I decided to apply mathematical morphology. In particular, I took advantage of the *binary closing* operation. The following algorithm implements this fix.

Algorithm 2: LV Segmentation

Input: Myocardium mask $\mathcal{M} = \{0, 1\}^{H \times W \times C}$
Output: LV mask $\mathcal{L} = \{0, 1\}^{H \times W \times C}$

- 1: **for** $z \leftarrow 1$ **to** C **do**
- 2: $\mathcal{M}_z \leftarrow \mathcal{M}[:, :, z]$
- 3: $F \leftarrow \text{binary_fill_holes}(\mathcal{M}_z) \oplus \mathcal{M}_z$
- 4: $r \leftarrow 3$
- 5: $\mathcal{M}'_z \leftarrow \mathcal{M}_z \bullet \mathcal{D}_r$ $\triangleright$ Morphological closing with disk of radius r
- 6: **if** $F \neq 0$ **or** $\sum(\mathcal{M}_z \oplus \mathcal{M}'_z) \leq 1$ $\triangleright$ If filling was successful or there was nothing to fill
- 7: **then**
- 8: $\mathcal{L}_z \leftarrow F$
- 9: **continue**
- 10: **while** $F = 0$ **and** $r < 20$ $\triangleright$ Filling failed and radius threshold was not met
- 11: **do**
- 12: $r \leftarrow r + 1$
- 13: $\mathcal{M}'_z \leftarrow \mathcal{M}_z \bullet \mathcal{D}_r$
- 14: $F \leftarrow \text{binary_fill_holes}(\mathcal{M}'_z) \oplus \mathcal{M}'_z$
- 15: $\mathcal{L}_z \leftarrow \text{binary_fill_holes}(\mathcal{M}'_z) \oplus \mathcal{M}_z$
- 16: **return** $\mathcal{L}_1 \cup \dots \cup \mathcal{L}_C$

The algorithm above yields an average dice score of 99.999% on the training data. Figure 4 shows the subjects and slices where the LV segmentation did not result in a 1.0 dice score.

6 Selected models

Once all features were extracted, it was time to train the models. Firstly, I ran a Cross-Validation Grid Search with a Random Forest Classifier. From the 100 subjects in the training data, I split them as 70 for training, 20 for global validation and 10 for testing. As such, the Cross-Validation was performed with 7 folds (60 data points for training and 10 for validation) only in the training set. I also opted for

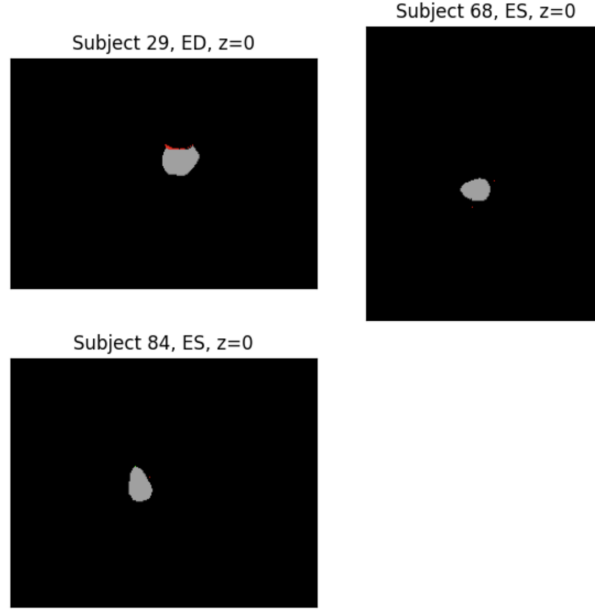


Figure 4: Subjects where the LV segmentation was not perfect. Pixels in red indicate false positives and pixels in green are missing in the segmentation.

maximization of macro-recall instead of accuracy. Although the challenge’s main metric is accuracy, I believe that in medical scenarios, it is best to optimize recall, for the consequences of miss-classifying someone as healthy are very high.

As previously said, the arguments that I varied on the Grid Search were the number of Decision Trees (`n_estimators`) and the maximum depth of each tree (`max_depth`). In particular, the values allowed to these arguments were as follows:

- `n_estimators` $\in \{100, 500, 1000, 2000\}$;
- `max_depth` $\in \{2, 3, 4, 5, 6\}$.

Despite Khened *et. al* setting the number of DTs to 100 [4], I increased the number of possible estimators due to the number of features used in this first model (thrice that of Khened’s model). Since Random Forests select $\sqrt{M}$ features per tree (if there are M features), it makes sense to allow for at most $\lceil \sqrt{M} \rceil$ levels in the trees to avoid overfitting. Because we have $M = 30$, I set the maximum `max_depth` to $\lceil \sqrt{30} \rceil = \lceil 5.477 \rceil = 6$. A depth of $\lceil \sqrt{M} \rceil$ was chosen as a rule-of-thumb, providing a reasonable trade-off between model complexity and interpretability. This was the first model chosen for evaluation.

Surely, training a model with 30 features is not going to yield the best results. We can see that the current literature supports a much lower number of features. Thus, I used this first model to rank all the features based on their importance. From this ranking, I was able to disconsider all features whose importance were below 0.01, using the `SelectFromModel` class from `sklearn`. The choice of filtering out features with less than 0.01 of importance was purely empirical.

Then, with the remaining features, I trained another RF in a Cross-Validation with Grid Search. This time, I varied the arguments as below:

- `n_estimators` $\in \{50, 80, 100, 120, 150, 512, 1000\}$;
- `max_detph` $\in \{2, 3, 4, 5, 6\}$.

The resulting model was chosen as the second for evaluation.

7 Results

After training the two models, the validation set was used to check their performance. Various training rounds were performed using different parameters or thresholds. Here, we highlight the performance of the chosen model on the validation data. The result can be seen in Figures 5 and 7. The best

hyperparameters found for model 1 were `n_estimators = 500` and `max_depth = 3`. For model 2, they were `n_estimators = 50` and `max_depth = 3`. Notably, Figure 6 shows the features importance identified by the first model. Similarly, the results on test data can be seen in Figures 8 and 9.

Overall, an accuracy of 90% was obtained, so that is what was expected on the private test data on Kaggle. The 100% accuracy on public leaderboard reinforced this belief, but in the end, the end accuracy was 82% with the second model.

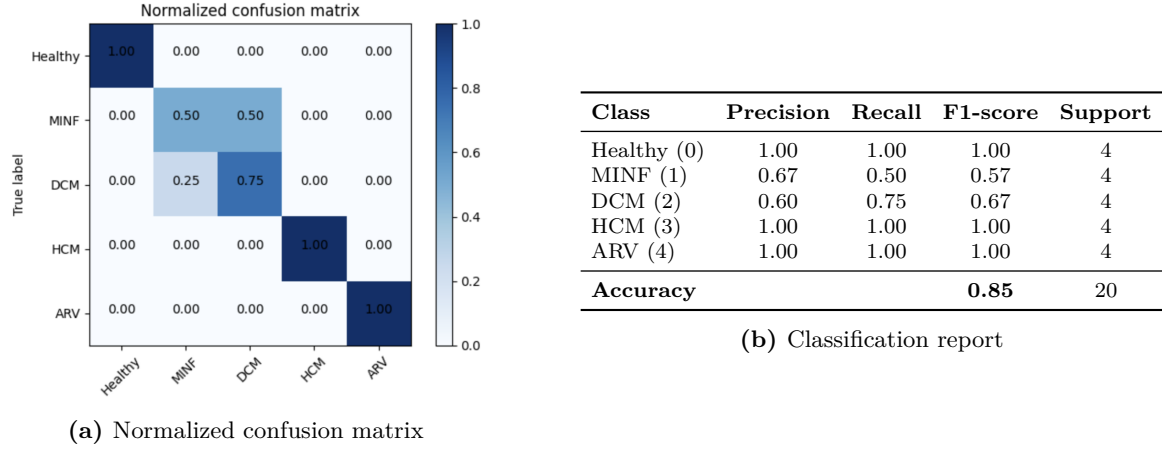


Figure 5: Model 1 validation results.

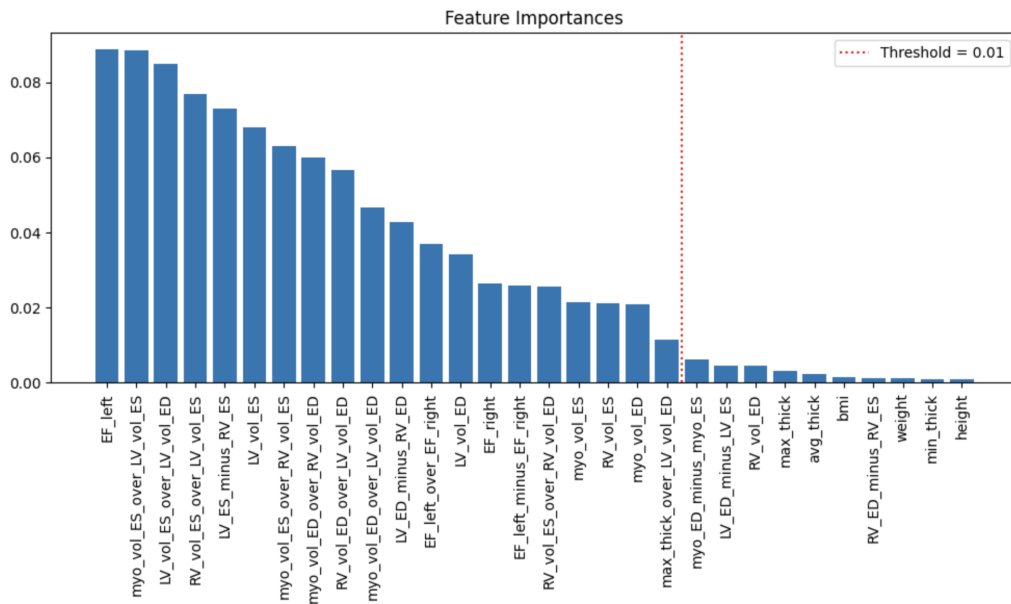
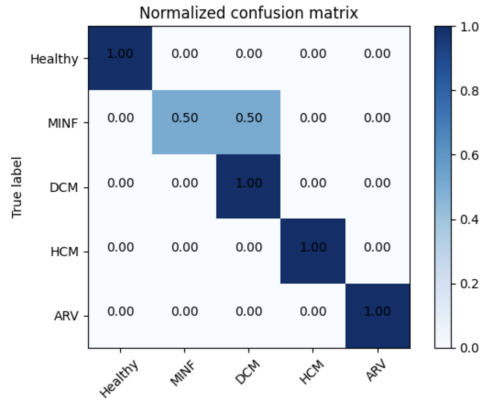


Figure 6: Feature importance yielded by model 1. The dotted red line separates the features that were used by model 2.

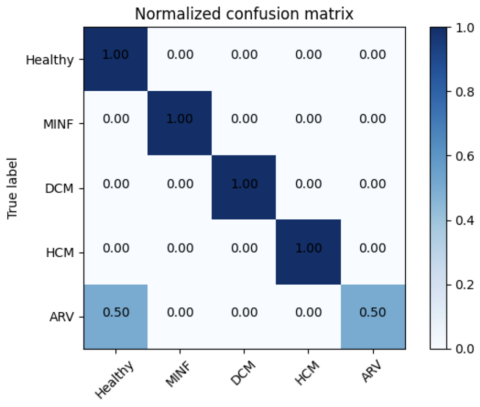


(a) Normalized confusion matrix

Class	Precision	Recall	F1-score	Support
0 (Healthy)	1.00	1.00	1.00	4
1 (MINF)	1.00	0.50	0.67	4
2 (DCM)	0.67	1.00	0.80	4
3 (HCM)	1.00	1.00	1.00	4
4 (ARV)	1.00	1.00	1.00	4
Accuracy	0.90			20

(b) Classification report

Figure 7: Model 2 validation results.

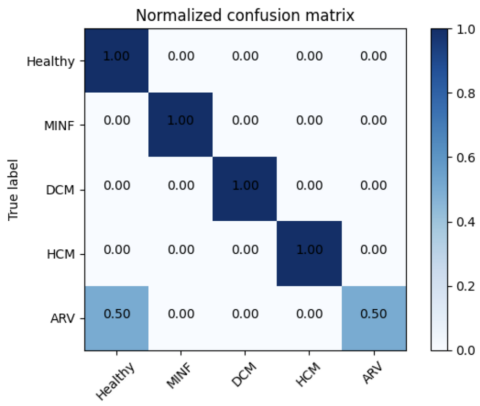


(a) Normalized confusion matrix

Class	Precision	Recall	F1-score	Support
0 (Healthy)	0.67	1.00	0.80	2
1 (MINF)	1.00	1.00	1.00	2
2 (DCM)	1.00	1.00	1.00	2
3 (HCM)	1.00	1.00	1.00	2
4 (ARV)	1.00	0.50	0.67	2
Accuracy	0.90			10

(b) Classification report

Figure 8: Model 1 test results.



(a) Normalized confusion matrix

Class	Precision	Recall	F1-score	Support
0 (Healthy)	0.67	1.00	0.80	2
1 (MINF)	1.00	1.00	1.00	2
2 (DCM)	1.00	1.00	1.00	2
3 (HCM)	1.00	1.00	1.00	2
4 (ARV)	1.00	0.50	0.67	2
Accuracy	0.90			10

(b) Classification report

Figure 9: Model 2 test results.

8 Conclusion

The model achieved a good overall performance with 82% accuracy on the 50 test subjects. The 8% accuracy difference between the testing set from the labeled dataset and Kaggle's test dataset may suggest overfitting of the model. This is likely the result of using 30 features in the first model and 20

in the second. However, the number of trees used by the best model 2 is troubling: 50 is little compared to Khened or Wolterink’s 100 and 1000 trees. So, it is possible that *underfitting* happened in training due to the lack of sufficient data points.

Instead of engineering more features, a set of fewer but better features might have yielded more robust results. For instance, the Body Surface Area (BSA) could have been used to normalize some of the volumes of the structures identified by the segmentation [1]. It would also be possible to increase the threshold of features importance to remove all features whose importance is below 0.05 or 0.1.

Interestingly, most errors in training and validation occurred between classes MINF (Myocardial infarction) and DCM (Dilated cardiomyopathy), while in the test set the only mistake took place between Healthy and ARV (Abnormal right ventricle). This may have been caused by the removal of the feature describing the volume of the RV at ED, although it had little importance (below 0.01).

Lastly, I recognize more could have been done when analyzing the features proposed. Plotting some essential graphs, like the correlation between features, could avoid some added noise in the DTs. Moreover, instead of choosing features by their importance, PCA could have been used to maximize the variability. Nevertheless, the intrinsic problem with PCA is that we would completely lose feature interpretability, so it is a double-edged sword.

References

- [1] BERNARD, O., LALANDE, A., ZOTTI, C., CERVENANSKY, F., YANG, X., HENG, P.-A., CETIN, I., LEKADIR, K., CAMARA, O., GONZALEZ BALLESTER, M. A., SANROMA, G., NAPEL, S., PETERSEN, S., TZIRITAS, G., GRINIAS, E., KHENED, M., KOLLERATHU, V. A., KRISHNAMURTHI, G., ROHÉ, M.-M., PENNEC, X., SERMESANT, M., ISENSEE, F., JÄGER, P., MAIER-HEIN, K. H., FULL, P. M., WOLF, I., ENGELHARDT, S., BAUMGARTNER, C. F., KOCH, L. M., WOLTERINK, J. M., IŞGUM, I., JANG, Y., HONG, Y., PATRAVALI, J., JAIN, S., HUMBERT, O., AND JODOIN, P.-M. Deep learning techniques for automatic mri cardiac multi-structures segmentation and diagnosis: Is the problem solved? *IEEE Transactions on Medical Imaging* 37, 11 (2018), 2514–2525.
- [2] FOLGOC, L. L., AND GORI, P. Ima205 challenge 2025. <https://kaggle.com/competitions/ima-205-challenge-2025>, 2025. Kaggle.
- [3] ISENSEE, F., JAEGER, P. F., FULL, P. M., WOLF, I., ENGELHARDT, S., AND MAIER-HEIN, K. H. Automatic cardiac disease assessment on cine-mri via time-series segmentation and domain specific features. In *Statistical Atlases and Computational Models of the Heart. ACDC and MMWHS Challenges* (Cham, 2018), M. Pop, M. Sermesant, P.-M. Jodoin, A. Lalande, X. Zhuang, G. Yang, A. Young, and O. Bernard, Eds., Springer International Publishing, pp. 120–129.
- [4] KHENED, M., ALEX, V., AND KRISHNAMURTHI, G. Densely connected fully convolutional network for short-axis cardiac cine mr image segmentation and heart diagnosis using random forest. In *Statistical Atlases and Computational Models of the Heart. ACDC and MMWHS Challenges* (Cham, 2018), M. Pop, M. Sermesant, P.-M. Jodoin, A. Lalande, X. Zhuang, G. Yang, A. Young, and O. Bernard, Eds., Springer International Publishing, pp. 140–151.
- [5] WOLTERINK, J. M., LEINER, T., VIERGEVER, M. A., AND IŞGUM, I. Automatic segmentation and disease classification using cardiac cine mr images. In *Statistical Atlases and Computational Models of the Heart. ACDC and MMWHS Challenges* (Cham, 2018), M. Pop, M. Sermesant, P.-M. Jodoin, A. Lalande, X. Zhuang, G. Yang, A. Young, and O. Bernard, Eds., Springer International Publishing, pp. 101–110.